Assignment "Object Oriented and Functional Programming with Python (DLBDSOOFPP01)”

Phase II

Course of Study: Data Science
Author: Soeren Olischer
mtknr: 9191235

Outline:

1. Introduction
2. Design
    2.1 Project Structure
    2.2 Sub Packages
        2.2.1 CLI
        2.2.2 SRC
    2.3 API Strategy
    2.4 Test Suit
3. Implementation
    3.1 SRC
        3.1.1 Analyze Functionality
        3.1.2 Habit Functionality
    3.2 Data
    3.3 CLI
4 Changes From The Initial Concept
    4.1 Changes In The SRC Packages
    4.2 Changes In The CLI Packages
5 Future Expectations
6 Tools Used
7 License
8 Installation
    8.1 General Requirements
    8.2 Virtual Python Environment
    8.3 Clone The Github Repository
    8.4 Installation With Pip
9 Usage
10 Semi Interactive Exploration Of The Tool

1. Introduction
this document represents the submit of the second phase of the portfolio for the module "Object Oriented and Functional Programming with Python"

The tool created throughout the implementation is called "Habtrack", it is fully implemented and able to manage and track habits as asked in the task

In the course of this document it will be represented how the tool was designed and how it was implemented.

It will be shown which tools were used for version management and which concepts and design patterns have found its way into this project

A short introduction to the usage of the tool will conclude this document

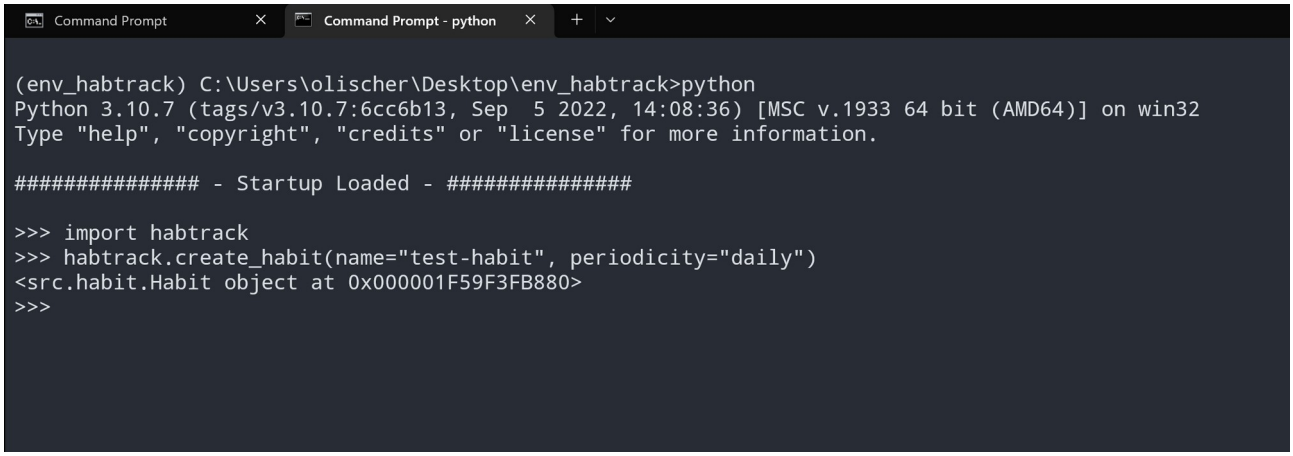
In addition to this document, the project is hosted on a GitHub repository:

https://github.com/077479/habit_tracker

and can be publicly accessed and cloned as well as used within the terms of the MIT License

2. Design
2.1 Project Structure
The project is provided as an importable python package as well as a usable command line tool. In order to achieve that the different functionality like the "program-logic" and the "command-line-interface" are separated into own sub-packages within the package.

The "__init__.py" file of the Package maps the "program-logic" to convenient api calls in order to use the package as importable "Python Package"



the pytest.ini file to configure the test-runs with pytest is also stored in the project root

An entry point file called habtrack.py within the project root folder serves as entry point for the usage as cli-tool

```
(env_habtrack) C:\Users\olischer\Desktop\env_habtrack>habtrack --demo list_habits
stored habits:
    Habit: work | 2000-01-01 | daily | amount: 1
    Habit: eat | 2000-01-01 | daily | amount: 2
    Habit: swimming | 2000-01-01 | weekly | amount: 1
    Habit: vacuum_clean | 2000-01-01 | weekly | amount: 3
    Habit: pay_rent | 2000-01-01 | monthly | amount: 1
    Habit: shop | 2000-01-01 | monthly | amount: 4

(env_habtrack) C:\Users\olischer\Desktop\env_habtrack>
```

2.2 Sub Packages

2.2.1 CLI

The sub-package CLI provides the Command-Line-Interface functionality, this subpackages also hosts the hardwired information for the help calls and the test-demo mode, aswell as the data for the semi-interactive-exploration tour of the tool

2.2.2 SRC

the sub-package source provides the actual buisness logic with all the modules for the objects and top level function in order to manage and track habits

2.3 API Strategy

the public api's of the different packages are not mapped to convenience calls, if its necessary to access the underlying logic he different modules have to be imported by themselves

2.4 Test Suit

The test strategy follows the principle "Many Unit-Tests" few "Functionality/E2E" tests.
The test suit of the project was initially held outside the project_root but were moved into the package in order to provide the tests for other users and or developers.
The test suit is also separated into different packages according to the division of the functionality of the tool itself. On top of that in the tests-root the functionality tests are stored, as well as the "conftest.py" file in order to provide necessary functions/fixtures

3 Implementation

3.1 SRC

3.1.1 Analyze Functionality

the analyze functionality is completely provided by the analyse module in the src sub-package, the functionality is implemented as a set of top-level-functions that are accessible by import
The core of this module is the top-level function get-streaks that returns the different streaks of a habit. the workflow of the function is:
to take a given habit,
copy the list of checkoffs from it (the list will be altered in the course of the process),
then the first time-period to check the check_offs against is created according to the creation date of the habit,
the valid check_offs of the habit are stored and removed from the copied check_offs list and the time_period is iterated and the process is repeated until the list of time_periods is empty

3.1.2 Habit Functionality

the habit functionality (the part that represents the actual habit with all his behavior and data) is broken down into two modules, a "period" module that provides a period-class to represent the periodicity and a "habit" module

the periodicity is designed to allow for additional time period is to be added according to the needs of an user, for that the functionality to provide the time-period to check against is stored as functionality that just increments a given date, through "monkey_patching" additional functionality can be added.

The design of the period object is inspired by the "state design pattern" where as a "hollow" object changes its behavior accordingly to the state its in

The habit just provides what someone would expect from a habit, a name, the periodicity as period object, an amount that represents the time a habit has to be marked as done in order to be fulfilled and the creation date aswell as the list of check_offs

The dates are completely represented by the python built-in "datetime.date" class, and when stored, stored as number according to the ordinal of the date, the datetime module provides convenient functionality that is implemented in a resource friendly way that an own implementation of a date was seen as unintentional

The underlying functionality that stores and retreives the stored data is provided by the "storage" module

3.2 Data
The data directory of the project hosts the actual stored habits as well as the demo-mode habits as "json" files, all actual habits are stored in a file called "habtrack.json", where as the demo-mode habits are stored in a file called "sample.json"

a habit is represented in json according to the following scheme:
```
{
"_Habit":true,
"name":string,
"description":string,
"periodicity":string,
"amount":int,
"creation_date":int,
"checkoffs": []
}
```

3.3 CLI
The command line interface is completly provided by this package, the general design is similar to the state pattern used to represent the periodicity, whereas in this package the different states are represented by different objects in different modules, to facilitate the different tasks each "state" has to access from the program-logic

to achieve that the entry point here is a module called "cli_start" which checks the to the command given argument for the "demo-flag", and then routes the first given argument as command according to the state that is needed.

This design idea is inspired by tools like git

The tool uses the built-in argparse module to process any given arguments, it was decided against the third-party module "click" because the desired git like usage is better created without the design path that clicks enforces, another considered option was to take the argument provided to the tool directly from the sys.argvar variable, but the parser that the python built-in module provides fits the needs of the desired design without complicated changes to it

the different states that access the underlying program-logic are designed as objects that inherit from a general command class so that the only implementation that the actual states need are the logic that is needed in order to fulfill their task without multiplying the same code to parse the arguments further over and over again

in addition to that a semi-interactive-exploraton is built in the module "manual", the functionality is implemented as top-level domain, it loads the complete manual for the cli_data module and processes it as generator, whereas the yielded information from the generator is a tuple that provides on the one side the needed "manual" data and on the other side the "cli calls" that are needed in order to show the gist of the tool

it was decided against a complete exploration of every functionality that the tool provides in favor of a brief introduction of the general usage of the tool. with the implemented cli-call design a potential user is completely able to understand the tool and to use it with the information the hardwired built-in reference provides.

the idea to this manual is inspired by a talk of Daniel Procida (core dev of django) during the PyCon 2017, he stated that the way a new user approaches a tool is through the documentation and that the documentation should be created in a way that animates the user to explore the tool but with the safety net of a reference that will help and guide the user through his expierence

4 Changes From The Initial Concept

Unfortunately the initial design had different design flaws that only were uncovered during the Implementation-Phase of the project, therefor it was necessary to differ from the, through the analyse/design phase created, initial concept. But the differences are minor according to the size of the project

4.1 Changes In The SRC Package

- src.habit.Habit
    - attribute checkoffs:
        changed from set to list, in order to allow multiple checkoffs for one day in order to allow habits that needs more than one checkoof a day
    - attribute amount_checkoffs:
        moved from src.period.Period to src.habit.Habit because everytime the amount of checkoffs is acessed an extra import had to be made, whereas everytime the amount of checkoffs is needed the habit object is there in the first place
    - added method change_periodicity:
        same as above everytime the periodicity of the habit object has to be changed the habit object is there, and with that change the period class does not need to be imported extra

4.2 Changes In The CLI Package

- added the option "a" for amount

- removed the possibility to manual deserialize, because the deserialization happens at program start

5 Future Expectation

- src.period.Period:
    - add an "add_time_period" function in order to fix this monkey-buisness with the adding a new timeperiod =>
      add_time_period(self, periodicity: str, function: fct) -> None:
- src.analyse.get_streaks
    - the whole list in a list thing is not a good design decision, maybe a separate object should be introduced that manages lists of habit, maybe inherited from list

6 Tools Used

- git:
    the version management were handled with the tool git
- pytest
    to execute the test suit the frame work "pytest" was used, the output of pytest is favored in comparison to "unittest"
- build
    to create the installable the build front end "build" was used

7. License

For the distribution the MIT license was chosen, it allows the free use of the project, even a commercial use, but restricts the liability/warranty

8. Installation

8.1 General Requirements

to use the tool the following requirements has to be met:

- python 3+
- git
- pip

8.2 Virtual Python Environment

the tool does not facilitate 3rd party modules, so there should not be any dependency issues, but for the sake of completeness the manual will go through the step of creating a virtual environment in order to install the tool.

```
$ python -m venv env_habit_tracker
$ cd env_habit_tracker

# Windows Only
$  .\Script\activate

#Linux / Mac
$ ./bin/activate
```

8.3 Clone the github Repository

the tool is hosted on github under the url: https://github.com/077479/habit_tracker.git
to clone the repository:

```
$ git clone https://github.com/077479/habit_tracker.git
```

this will copy the repository to a directory called habit_tracker.
From here it could be imported in python and used with the python interpreter, to use it as a CLI Tool like git it has to be installed

8.4 Installation with Pip

after the copy of the tool it can be installed with pip with the in the tool included wheel file

```
$ cd habit_tracker/wheel
$ pip install habtrack-1.0-py3-none-any.whl
```

Now the tool can be used like any oter CLI Tool installed

9 Usage

After the installation (steps above) the "habtrack" tool is aviable as shell command.
the general usage of the tool is:

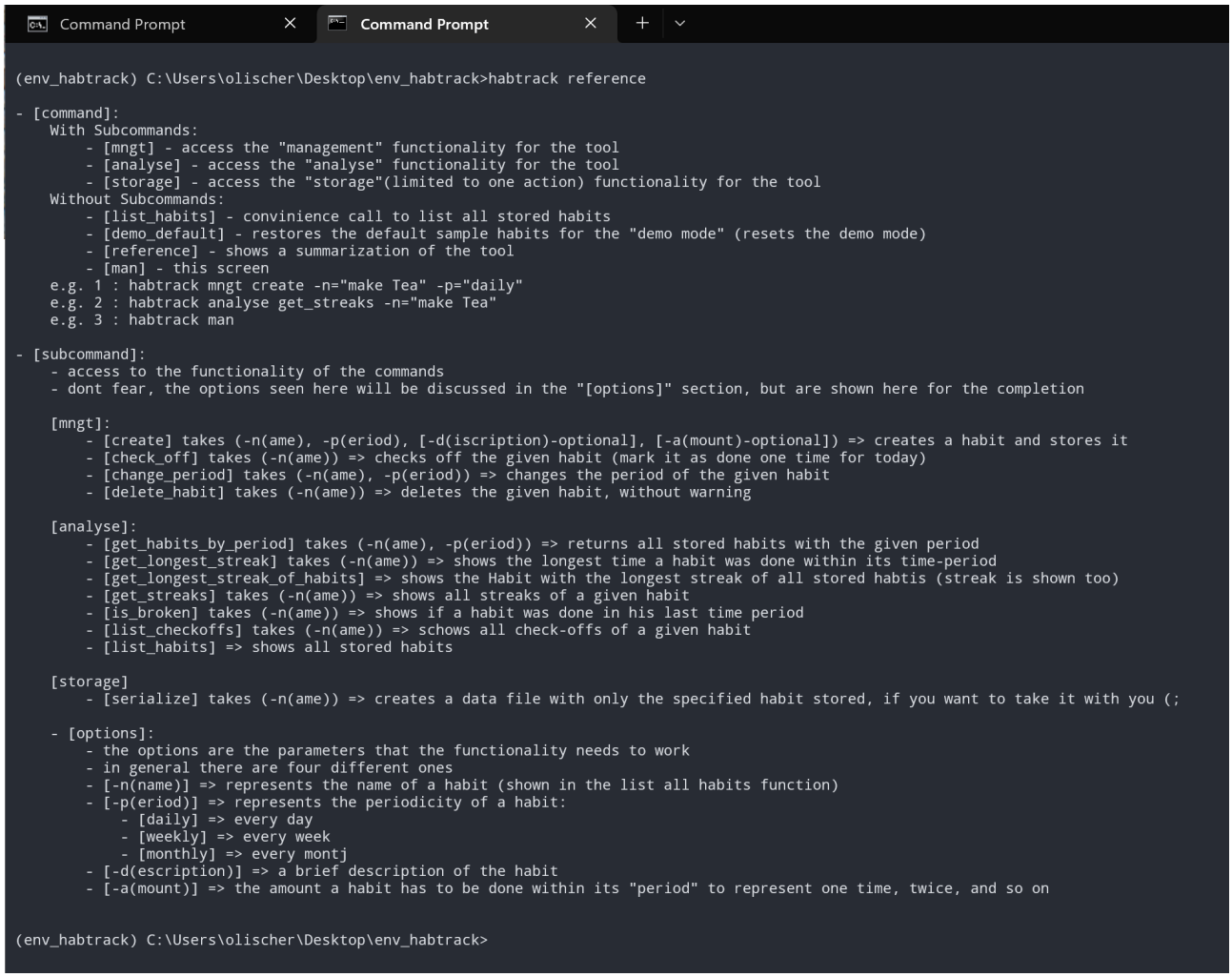
```
$ habtrack [command] [subcommand]
```

Every Command has a help option:

```
$ habtrack help
$ habtrack [command] help
$ habtrack [command] [subcommand] help
```

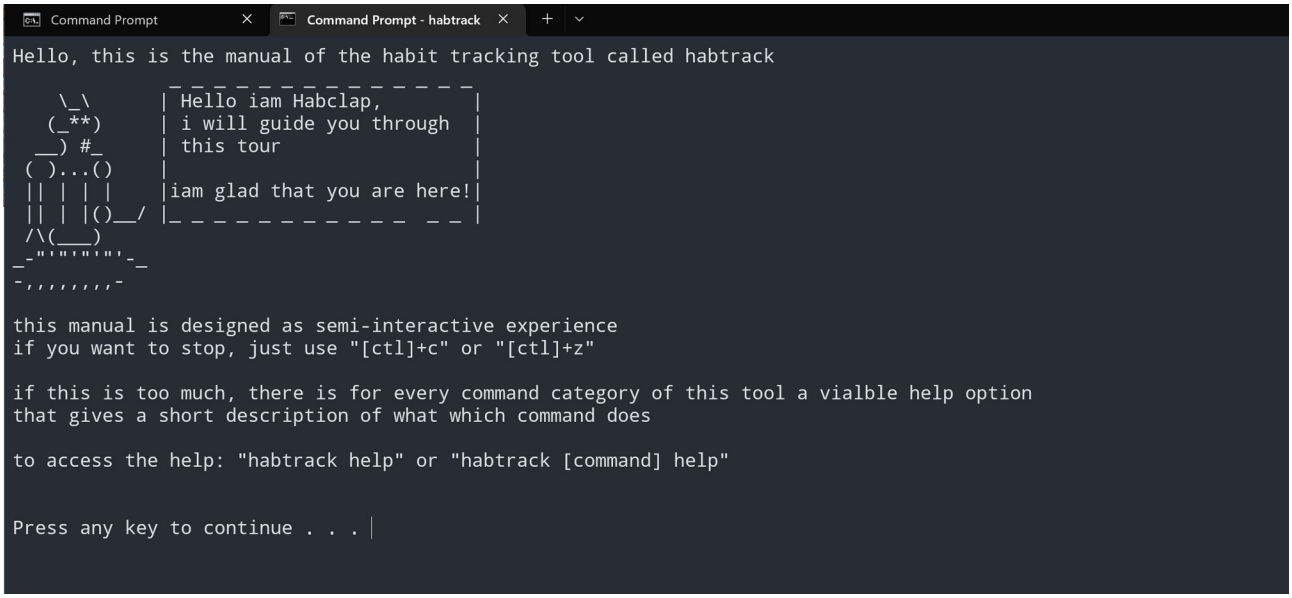
A reference to the usage of the tool can be accessed with:

```
$habtrack reference
```



```
Command Prompt    Command Prompt

(env_habtrack) C:\Users\olischer\Desktop\env_habtrack>habtrack reference

- [command]:
    With Subcommands:
        - [mngt] - access the "management" functionality for the tool
        - [analyse] - access the "analyse" functionality for the tool
        - [storage] - access the "storage"(limited to one action) functionality for the tool
    Without Subcommands:
        - [list_habits] - convinience call to list all stored habits
        - [demo_default] - restores the default sample habits for the "demo mode" (resets the demo mode)
        - [reference] - shows a summarization of the tool
        - [man] - this screen
    e.g. 1 : habtrack mngt create -n="make Tea" -p="daily"
    e.g. 2 : habtrack analyse get_streaks -n="make Tea"
    e.g. 3 : habtrack man

- [subcommand]:
    - access to the functionality of the commands
    - dont fear, the options seen here will be discussed in the "[options]" section, but are shown here for the completion

    [mngt]:
        - [create] takes (-n(ame), -p(eriod), [-d(iscription)-optional], [-a(mount)-optional]) => creates a habit and stores it
        - [check_off] takes (-n(ame)) => checks off the given habit (mark it as done one time for today)
        - [change_period] takes (-n(ame), -p(eriod)) => changes the period of the given habit
        - [delete_habit] takes (-n(ame)) => deletes the given habit, without warning

    [analyse]:
        - [get_habits_by_period] takes (-n(ame), -p(eriod)) => returns all stored habits with the given period
        - [get_longest_streak] takes (-n(ame)) => shows the longest time a habit was done within its time-period
        - [get_longest_streak_of_habits] => shows the Habit with the longest streak of all stored habtis (streak is shown too)
        - [get_streaks] takes (-n(ame)) => shows all streaks of a given habit
        - [is_broken] takes (-n(ame)) => shows if a habit was done in his last time period
        - [list_checkoffs] takes (-n(ame)) => schows all check-offs of a given habit
        - [list_habits] => shows all stored habits

    [storage]
        - [serialize] takes (-n(ame)) => creates a data file with only the specified habit stored, if you want to take it with you (;

    - [options]:
        - the options are the parameters that the functionality needs to work
        - in general there are four different ones
        - [-n(name)] => represents the name of a habit (shown in the list all habits function)
        - [-p(eriod)] => represents the periodicity of a habit:
            - [daily] => every day
            - [weekly] => every week
            - [monthly] => every montj
        - [-d(escription)] => a brief description of the habit
        - [-a(mount)] => the amount a habit has to be done within its "period" to represent one time, twice, and so on


(env_habtrack) C:\Users\olischer\Desktop\env_habtrack>
```

10 Semi Interactive Exploration of the Tool

Built in in this tool is a CLI-based semi interactive exploration of the tool where a ascii robot takes the user through a tour that explains the general use of the tool, which in combination with the reference should be sufficient to use the tool in all its functionality

```
$ habtrack man
```